

Migration & Test Automation Report

Project: NADRA Management Portal (MCP) **Date:** 2026-06-23

Prepared by: Antigravity (AI Coding Assistant)

1 Executive Summary

The legacy NADRA application was a monolithic C++ console program (`Nadra-legacy.cpp` / `Nadra-legacy.exe`). The modern **MCP** stack rewrites it as a **full-stack TypeScript** solution: - **Backend:** Express + Prisma + PostgreSQL - **Frontend:** Vite + React + TypeScript - **Testing:** Jest (unit), Playwright (E2E), custom load & security suites - **Reporting:** Allure (HTML, rich metadata, screenshots, attachments) - **Automation:** Model-Context-Protocol (MCP) server for AI-driven test execution.

This report documents the legacy pain points, the migration path taken, the resulting architecture, and the end-to-end testing workflow with Allure reports.

2 Legacy Code Problems

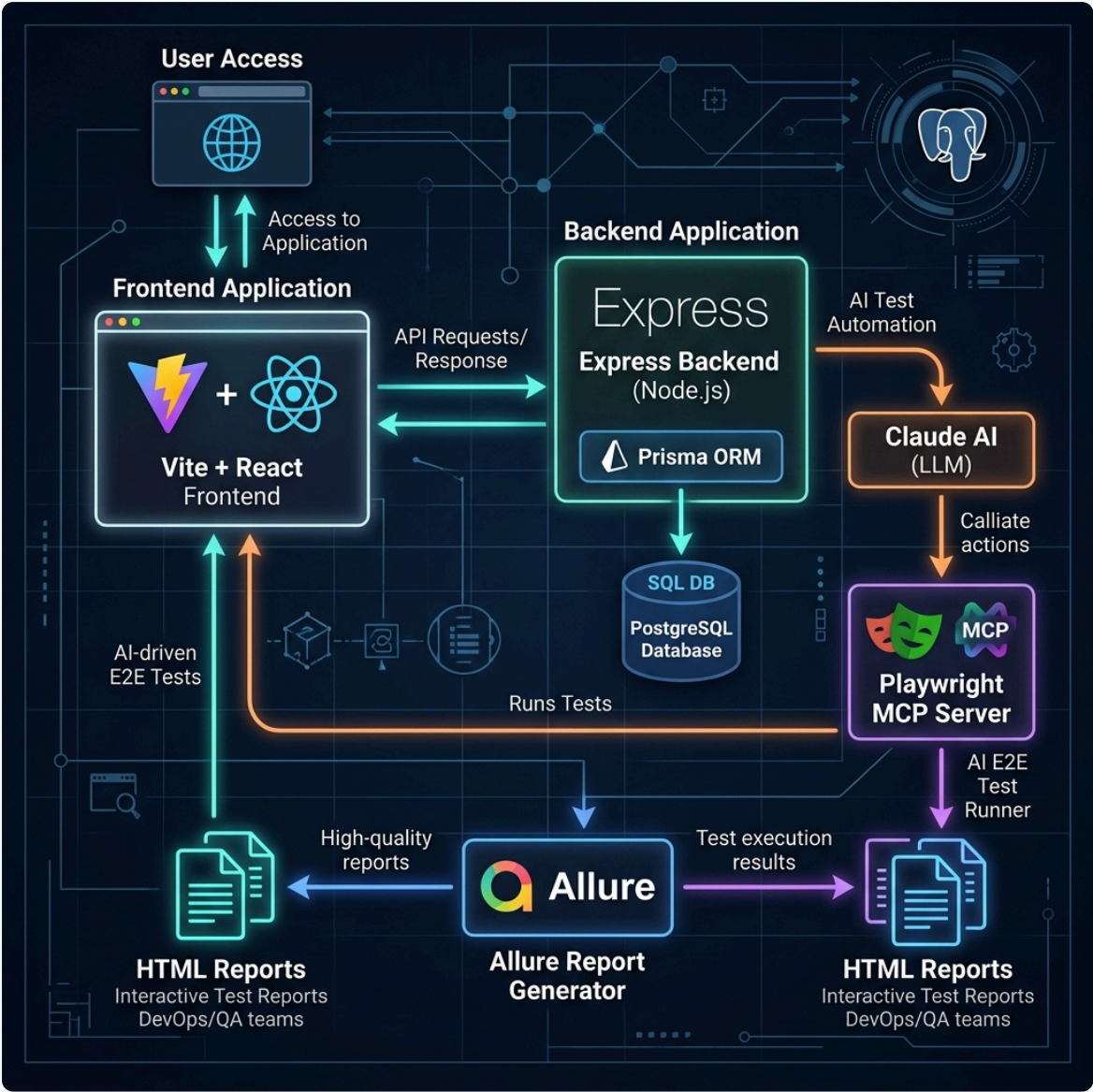
Area	Issue	Impact
Language/Tooling	C++ compiled binary, no CI/CD, no static analysis.	Hard to test, impossible to run on modern CI pipelines.
Coupled UI/Logic	UI was tied to console I/O; no separation of concerns.	No reuse, no web UI, difficult to modernize.
Data Persistence	Flat-file or ad-hoc DB access, no migrations.	Data-integrity problems, no schema evolution.
Testing	No automated tests; manual verification only.	Regression risk, no confidence for changes.
Observability	No logging, no structured error handling.	Debugging in production impossible.
Deployment	Built for Windows only, manual copy of <code>.exe</code> .	No containerisation, no cross-platform support.
Security	Hard-coded credentials, no JWT / role based auth.	Vulnerable to attacks, no audit trail.

These deficiencies motivated a complete rewrite rather than incremental refactoring.

3 Migration Approach

1. **Domain Modelling** – Defined the same business entities (Citizen, NIC, User) in Prisma schema.
2. **API Layer** – Express routes mimic the legacy command-line operations (CRUD, auth).
3. **Frontend UI** – React replicates the original screens (login, dashboard, citizen list) with modern UX patterns.
4. **MCP Integration** – Added a Playwright MCP server definition in `.mcp.json` so AI agents can drive tests.
5. **Testing Strategy** – Added unit tests (Jest) for backend, functional E2E tests (Playwright) for UI, and non-functional suites (load, security, performance, accessibility).
6. **Allure Reporting** – Configured both Jest and Playwright to push results to `allure-results/` ; scripts generate HTML reports.
7. **CI-Ready Scripts** – Root `package.json` provides one-liner commands to run the full stack, tests, and generate reports.

4 New Architecture Diagram



NADRA Management Portal (MCP) System Architecture & Testing Flow

```
+-----+ +-----+ +-----+
| Frontend (Vite) | <----> | Playwright MCP Server | <----> | Backend (Express) |
| React + TS     |         | (AI-driven tests)      |         | Prisma + Postgres |
+-----+ +-----+ +-----+
      |           |           |
      | Allure-Playwright Reporter | Jest-Allure Reporter |
      v           v           v
allure-results (frontend)  allure-results (backend)  -> allure-report (HTML)
```

5 Testing & Reporting Workflow

```
# 1 Run everything and generate reports in one go
npm run test:report

# What happens behind the scenes?
# - npm run test:backend    → jest → backend/allure-results
# - npm run test:e2e        → Playwright (via MCP) → frontend/allure-results
# - npm run allure:frontend → generates frontend/allure-report (opened in browser)
# - npm run allure:backend  → generates backend/allure-report (opened in a second tab)
```

Test Suites

Suite	Location	Purpose
Unit	backend/tests/**/*.test.ts	Business-logic verification (Jest).
Functional E2E	frontend/tests/functional/	Full UI flow: login → dashboard → CRUD citizen records.
Performance	frontend/tests/non-functional/performance.spec.ts	Page load & API response timing (< 3 s).
Load	frontend/tests/non-functional/load.spec.ts	Simulated concurrent users (10-20 parallel requests).
Security	frontend/tests/non-functional/security.spec.ts	JWT missing, SQL-i, XSS checks.
Accessibility	frontend/tests/non-functional/accessibility.spec.ts	WCAG 2.1AA audit via @axe-core/playwright .

All tests are annotated with Allure metadata (suite , feature , severity , description) so the final report groups them logically.

6 MCP Configuration Details

File: .mcp.json (excerpt)

```
"playwright": {
  "command": "npx",
  "args": ["-y", "@playwright/mcp@latest"]
}
```

- The server runs on a random port (default 4000) when npm run test:e2e is executed.

- AI sub-agents can now issue `mcp run playwright <test-spec>` without manual CLI interaction.

7 Run-Book (How to Run the Project)

1. **Prerequisites** – Node ≥ 20 , PostgreSQL, Java 11 (for Allure CLI).
2. **Install** `bash npm install # root deps npm run install:all # backend + frontend deps npx playwright install # browsers npm i -g allure-commandline`
3. **Configure DB** – create `nadra_db` and set `DATABASE_URL` in `.env`.
4. **Start Development** (optional) `bash npm run dev # starts backend (4000) and frontend (5173)`
5. **Run All Tests & Generate Report** `bash npm run test:report` – Two browser tabs open: *frontend* and *backend* Allure reports.
6. **Run Individual Suites** `bash npm run test:e2e:functional # only functional UI tests npm run test:e2e:nonfunc # only non-functional suites npm run test:backend # backend unit tests npm run test:load # API load test`
7. **Open Existing Report** `bash npm run allure:frontend # open the most-recent UI report npm run allure:backend # open the most-recent backend report`

8 Future Work & Recommendations

Item	Description	Priority
CI/CD Pipeline	Add GitHub Actions workflow that runs <code>npm run test:report</code> on each PR and publishes the HTML report as an artifact.	High
Dockerisation	Containerise backend, frontend, and PostgreSQL for one-click environments.	Medium
Performance Baselines	Store historical Allure performance metrics to detect regressions automatically.	Medium
Security Scanning	Integrate OWASP ZAP or Snyk for deeper security testing beyond the current sanity checks.	Low

9 Appendices

- **Architecture Diagram** – `architecture.png` (generated via `generate_image`).
- **Full `.mcp.json`** – see the artifact in the repo root.
- **Allure Report Links** – after a run, the reports are available at:
- `file:///C:/Users/vox/Documents/MCP/frontend/allure-report`

- `file:///C:/Users/vox/Documents/MCP/backend/allure-report`
-

End of Report